

Capítulo 03

Ngram Model

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 30/07/2026

Capítulo 03 — N-gram model (MLP em PyTorch)

Objetivo de aprendizagem: superar a memória curta do bigrama construindo um **MLP** (multi-layer perceptron) que olha **vários** caracteres de contexto. Pelo caminho, encontramos peças que vão aparecer no curso inteiro: **embeddings**, **matmul** em lote, a ativação **GELU**, **mini-batches**, **cross-entropy** e — o conceito mais importante de todos — a separação **treino / validação / teste**.

Pré-requisitos deste capítulo: Capítulos 1 (modelo de linguagem, softmax, loss) e 2 (como uma rede aprende por gradient descent). A partir daqui usamos **PyTorch de verdade** — a `backward()` que construímos à mão no Cap. 2 agora é a do framework, e sabemos exatamente o que ela faz.

Arquivos: - `prepare_data.py` — prepara o dataset de nomes (fonte: IBGE) - `mlp.py` — o modelo MLP completo (treino, avaliação e geração) - `exercicios.md` — exercícios

1. O problema do bigrama: memória de um caractere só

No Capítulo 1, o bigrama previa o próximo caractere olhando **apenas o anterior**. Ao gerar o nome `maria`, na hora de decidir a letra depois do segundo `a`, ele já tinha esquecido o `m`, o `r`... tudo. Uma memória de um caractere só.

A consequência: nomes "pronunciáveis", mas sem estrutura de longo alcance. Para melhorar, precisamos de **mais contexto** — olhar os últimos `k` caracteres em vez de um só. A ideia ingênua seria um **trigrama** (dois caracteres anteriores), depois um quadrigrama, e assim por diante.

Por que contar não escala

Lembra que no Cap. 1 a contagem virou uma matriz 27×27 ? Para um trigrama, ela vira $27 \times 27 \times 27$. Para `k` caracteres de contexto, são $27^{(k+1)}$ células. Com `k = 10`, isso passa de **10^{20} células** — impossível de armazenar, e a maioria ficaria zerada (nenhum nome tem aquela combinação específica). É a **maldição da dimensionalidade**: mais contexto, com contagem, significa uma explosão de células vazias.

A saída é exatamente a do Cap. 1, levada a sério: em vez de **contar**, vamos **aprender** — com uma rede neural que comprime o contexto num espaço pequeno e denso. Essa é a arquitetura de Bengio et al. (2003), e é a base de tudo que vem depois.

2. Um dataset de verdade

Há uma mudança importante em relação aos capítulos anteriores: trocamos os 155 nomes de brinquedo por um dataset **real e grande** — os prenomes dos censos do **IBGE**, com mais de **64 mil nomes**.

```
maria, ana, joao, gabriel, lucas, pedro, mateus, jose, gustavo, vitoria, ...
```

O `prepare_data.py` baixa a lista do IBGE e **normaliza** cada nome: minúsculo, sem acento (`josé -> jose`), só letras `a-z`. Para reproduzir:

```
curl -L -o _ibge.csv
https://raw.githubusercontent.com/datasets-br/prenomes/master/data/nomes-censos-ibge.csv
python prepare_data.py # gera names.txt
```

Por que isso importa tanto? Um MLP tem milhares de parâmetros. Com poucos dados (os 155 nomes do Cap. 1), ele simplesmente **decora** o dataset e não aprende nada generalizável — um problema chamado *overfitting*, que veremos de perto na Seção 8. Com dezenas de milhares de exemplos, ele é forçado a aprender **padrões** de verdade. O tamanho do dataset não é detalhe: é o que separa um modelo que funciona de um que só finge funcionar.

3. Construindo o dataset de contexto -> alvo

A janela de contexto tem tamanho fixo `block_size` (usamos `3`). Para cada posição do nome, o **input** são os 3 caracteres anteriores e o **alvo** é o caractere atual. Usamos o token de fronteira `.` (índice 0) para preencher o começo.

Veja como o nome `ana` é fatiado (`block_size = 3`):

```
contexto -> alvo
. . . -> a (começo do nome)
. . a -> n
. a n -> a
a n a -> . (fim do nome)
```

No código, a janela "desliza": a cada passo descartamos o caractere mais antigo e anexamos o atual.

```
def build_dataset(words):
    X, Y = [], []
    for w in words:
        context = [0] * block_size # '...'
        for ch in w + ".":
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix] # desliza a janela
    return torch.tensor(X), torch.tensor(Y)
```

`X` tem formato `(N, 3)` (`N` contextos, 3 índices cada) e `Y` tem formato `(N,)`.

4. A peça nova: embeddings

Como dar os caracteres de input para a rede? No Cap. 1 usamos **one-hot** (um vetor de 27 posições com um único `1`). Funciona, mas é desperdício: cada caractere fica isolado, sem relação com os outros.

A ideia melhor é o **embedding**: representar cada caractere por um vetor pequeno e **denso** de números reais — que a própria rede vai **aprender**. Guardamos isso numa tabela **C** de formato `(vocab_size, n_embd)`: uma linha por caractere, cada linha um vetor de `n_embd = 10` números.

```
C = torch.randn((vocab_size, n_embd)) # (27, 10): a tabela de embeddings
emb = C[X] # (N, 3, 10): "olha" o embedding de cada char
```

C[X] é indexação avançada do PyTorch. **X** tem os índices dos caracteres; **C[X]** substitui cada índice pela linha correspondente de **C**. Como **X** é `(N, 3)`, o resultado é `(N, 3, 10)`: para cada um dos **N** exemplos, os embeddings dos seus 3 caracteres de contexto. É o equivalente vetorizado e eficiente do "one-hot vezes matriz" do Cap. 1.

O bônus pedagógico: depois de treinar, caracteres com papéis parecidos (por exemplo, as vogais) tendem a ganhar embeddings próximos — a rede **descobre** estrutura sozinha. (Você explora isso no exercício E6.)

5. A arquitetura do MLP

Com os embeddings em mãos, o resto é uma rede de duas camadas:

```
def forward(X):
    emb = C[X] # (N, 3, 10)
    x = emb.view(emb.shape[0], -1) # (N, 30): concatena o contexto num vetor só
    h = F.gelu(x @ W1 + b1) # (N, 200): camada oculta com GELU
    logits = h @ W2 + b2 # (N, 27): pontuações do próximo caractere
    return logits
```

Três operações merecem atenção:

emb.view(N, -1) — **achatar o contexto**. A camada linear espera um vetor por exemplo, não uma matriz 3×10 . O `.view` reorganiza os mesmos números em `(N, 30)`, colando os 3 embeddings de 10 em um vetor de 30. (O `-1` diz ao PyTorch "calcule essa dimensão pra mim".) `view` não copia dados — é o conceito de *views/strides* do apêndice; só reinterpreta o mesmo bloco de memória.

x @ W1 + b1 — **a matmul**. O `@` é a **multiplicação de matrizes**: combina os 30 números de entrada nos 200 neurônios da camada oculta, de uma vez, para todo o batch. É a operação que domina o custo de qualquer rede neural — e o motivo de usarmos GPUs (Capítulo 8).

F.gelu(...) — **a não-linearidade**. Como vimos no Cap. 2, sem uma função não-linear a rede inteira colapsaria numa única transformação linear. Aqui usamos a **GELU** (*Gaussian Error Linear Unit*) no lugar da `tanh`: ela é a ativação padrão dos Transformers modernos (GPT, BERT). Na prática é uma versão "suave" do ReLU — deixa passar valores positivos, amortece os negativos sem zerá-los bruscamente.

6. A loss: `cross_entropy`

No Cap. 1 calculamos a loss em três passos manuais: softmax, pegar a probabilidade do alvo, tirar `-log` e a média. O PyTorch faz tudo isso de uma vez:

```
loss = F.cross_entropy(logits, Y)
```

`cross_entropy` é softmax + negative log-likelihood combinados. Usá-la em vez de fazer à mão não é só conveniência: a versão do PyTorch é **numericamente estável** (evita `exp()` de números grandes que estourariam) e mais rápida. É a loss padrão de todo modelo de classificação — e prever "qual o próximo entre 27 caracteres" é um problema de classificação.

7. Treino com mini-batches

Nosso dataset de treino tem **mais de 400 mil** exemplos. Calcular a loss sobre todos a cada passo seria lento. A solução é o **mini-batch**: a cada passo, sorteamos um punhado de exemplos (32) e damos um passo de gradiente baseado só neles.

```
for step in range(max_steps):
    ix = torch.randint(0, Xtr.shape[0], (batch_size,)) # 32 índices aleatórios
    logits = forward(Xtr[ix])
    loss = F.cross_entropy(logits, Ytr[ix])

    for p in parameters:
        p.grad = None
        loss.backward()

    lr = 0.1 if step < 15000 else 0.01 # learning rate decai no fim
    for p in parameters:
        p.data += -lr * p.grad
```

Duas observações:

- **O gradiente do mini-batch é "ruidoso"** (é uma estimativa, baseada em 32 exemplos, do gradiente verdadeiro). Por isso a loss impressa **pula** de um passo para outro — é normal e até útil: o ruído ajuda a escapar de mínimos ruins. O que importa é a tendência de queda, não cada valor.
- **O decaimento da learning rate** (de `0.1` para `0.01` no fim) é o truque de "passos largos no começo, passos finos para assentar". Inicialização e learning rate ganham um capítulo só pra eles (Cap. 7); aqui usamos valores razoáveis.

8. O conceito mais importante: treino / validação / teste

Aqui está a ideia que diferencia "decorar" de "aprender". Se medíssemos a qualidade do modelo **nos mesmos dados** em que ele treinou, um modelo que simplesmente **decorou** todos os nomes teria loss baixíssima — e seria inútil para nomes novos.

Por isso dividimos os dados em **três** partes:

Split	Proporção	Para que serve
treino	80%	ajustar os pesos (o modelo vê esses dados)
validação	10%	medir generalização e escolher hiperparâmetros
teste	10%	a prova final, usada uma vez só, no fim

```
random.shuffle(words)
n1, n2 = int(0.8*len(words)), int(0.9*len(words))
Xtr, Ytr = build_dataset(words[:n1]) # treino
Xdev, Ydev = build_dataset(words[n1:n2]) # validação
Xte, Yte = build_dataset(words[n2:]) # teste
```

A pergunta-chave: a loss de **validação** acompanha a de **treino**? Se a de treino despenca mas a de validação não acompanha (ou piora), o modelo está **decorando** em vez de generalizar — isso é o **overfitting**.

Veja na prática. O dataset grande deste capítulo dá treino e validação quase coladas (Seção 9) — sinal de generalização saudável. Se você rodar o **mesmo código** com os 155 nomes do Capítulo 1 (exercício E5), verá a loss de validação **disparar** enquanto a de treino cai: o retrato clássico do overfitting. Mesmo modelo, dataset pequeno — eis a diferença.

9. Rodando e os resultados

Rode `python mlp.py`. A saída (com a semente fixa do código):

```
treino: (410556, 3) | val: (51615, 3) | teste: (51504, 3)
parametros: 11897
step 0/20000 | loss 3.3753
step 2000/20000 | loss 2.2718
...
loss treino = 1.9659
loss validacao = 1.9666
loss teste = 1.9740
```

Dois pontos a comemorar:

1. **Treino $\approx$ validação $\approx$ teste ($\sim 1,97$).** As três losses praticamente coincidem — o modelo **generaliza**. O que ele aprendeu nos 80% de treino vale igualmente para nomes que nunca viu.
2. **$\sim 1,97$ é bem melhor que o bigrama.** O bigrama do Capítulo 1 ficava em $\sim 2,4$ de loss. O MLP, com mais contexto, chega a $\sim 1,97$. **Mais contexto + capacidade de aprender = modelo melhor.** A tese do capítulo, confirmada por um número.

E os nomes gerados ficam mais variados e estruturados que os do bigrama:

```
nan, ven, jelia, maurisney, alemyr, mara, clesilenna, ...
```

Note que são nomes **novos** (não estão no dataset) e ainda assim plausíveis em português — exatamente o que queremos.

10. Resumo do capítulo

- O bigrama tem memória de **um** caractere; contar não escala para mais contexto (maldição da dimensionalidade). A saída é **aprender** com um MLP.
- **Embedding**: cada caractere vira um vetor denso e aprendível ($C[X]$), bem melhor que one-hot.
- O **MLP** concatena os embeddings do contexto (**view**), aplica uma camada oculta com **GELU** ($x @ w1 + b1$) e produz **logits** ($h @ w2 + b2$).
- **cross_entropy** = softmax + NLL, estável e padrão.
- **Mini-batches** tornam o treino viável em datasets grandes (loss "ruidosa" é normal); a **learning rate** **decai** para assentar no fim.
- **Treino / validação / teste**: a única forma honesta de medir se o modelo **generaliza** ou só **decora** (overfitting). Dataset grande -> generalização.
- Resultado: loss ~1,97, **batendo o bigrama** (~2,4), com nomes gerados melhores.

O que vem no Capítulo 4

O MLP ainda trata o contexto como um saco fixo de 3 caracteres concatenados. E se o modelo pudesse decidir **a quais** caracteres do passado prestar atenção, de forma flexível e dependente do conteúdo? Esse é o salto do **Capítulo 04 — Attention**, o mecanismo que torna os Transformers tão poderosos.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 03 (N-gram model / MLP)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

E1 — Leitura de código (aquecimento)

Sem rodar, responda: 1. Por que `emb = C[X]` tem formato `(N, 3, 10)`? O que significa cada uma das três dimensões? 2. Por que precisamos do `.view(N, -1)` antes da primeira camada linear? 3. O que `F.cross_entropy(logits, Y)` faz, em termos do que fizemos "à mão" no Capítulo 1?

E2 — O efeito do contexto (`block_size`)

Rode `mlp.py` com `block_size = 1, 3 e 5` (três execuções). 1. Anote a loss de validação em cada caso. Mais contexto ajuda? 2. Com `block_size = 1`, o MLP vira essencialmente um bigrama "neural". A loss fica parecida com a do Capítulo 1 (~2,4)? Por quê? 3. O número de parâmetros muda com o `block_size`? Onde, no código, isso aparece?

E3 — Tamanho da camada oculta

Varie `n_hidden` em 50, 200 e 500. 1. Como muda o número de parâmetros e a loss de validação? 2. Camada maior sempre dá loss menor? Onde parece haver um retorno decrescente?

E4 — Learning rate

1. Troque a learning rate inicial (`0.1`) por `1.0` e por `0.001`. Descreva o que acontece com a curva de loss em cada caso (instável? lenta?).
 2. Remova o decaimento (deixe `lr = 0.1` o tempo todo). A loss final piora? Por que o decaimento ajuda no fim do treino?
-

E5 — Overfitting com dataset pequeno (o mais importante)

Copie os **155 nomes** do Capítulo 1 para um arquivo `names_pequeno.txt` e rode o MLP com ele (mude o nome do arquivo no código, ou use a solução pronta). 1. O que acontece com a loss de **treino** versus a de **validação**? 2. Por que o **mesmo modelo** generaliza com 64 mil nomes mas decora com 155? 3. Os nomes "gerados" passam a se parecer demais com nomes reais do dataset. Por quê isso é um **mau** sinal, e não um bom?

Solução de referência: [solucoes/e5_overfitting.py](#).

E6 — Visualizando os embeddings (desafio)

Após treinar, a tabela `C` tem um vetor por caractere. Para "ver" o que a rede aprendeu, treine com `n_embd = 2` (assim cada caractere é um ponto no plano) e plote os 27 caracteres com `matplotlib`. 1. As **vogais** (a, e, i, o, u) ficam agrupadas? E outros grupos fazem sentido? 2. Por que reduzir para `n_embd = 2` piora a loss, mas ainda assim é útil para visualizar?

E7 — Conte os parâmetros na mão (desafio)

O código imprime `parametros: 11897`. Mostre, somando as partes (`C`, `w1`, `b1`, `w2`, `b2`), de onde vem esse número — em função de `vocab_size`, `block_size`, `n_embd` e `n_hidden`. Confira que sua fórmula reproduz o 11897.